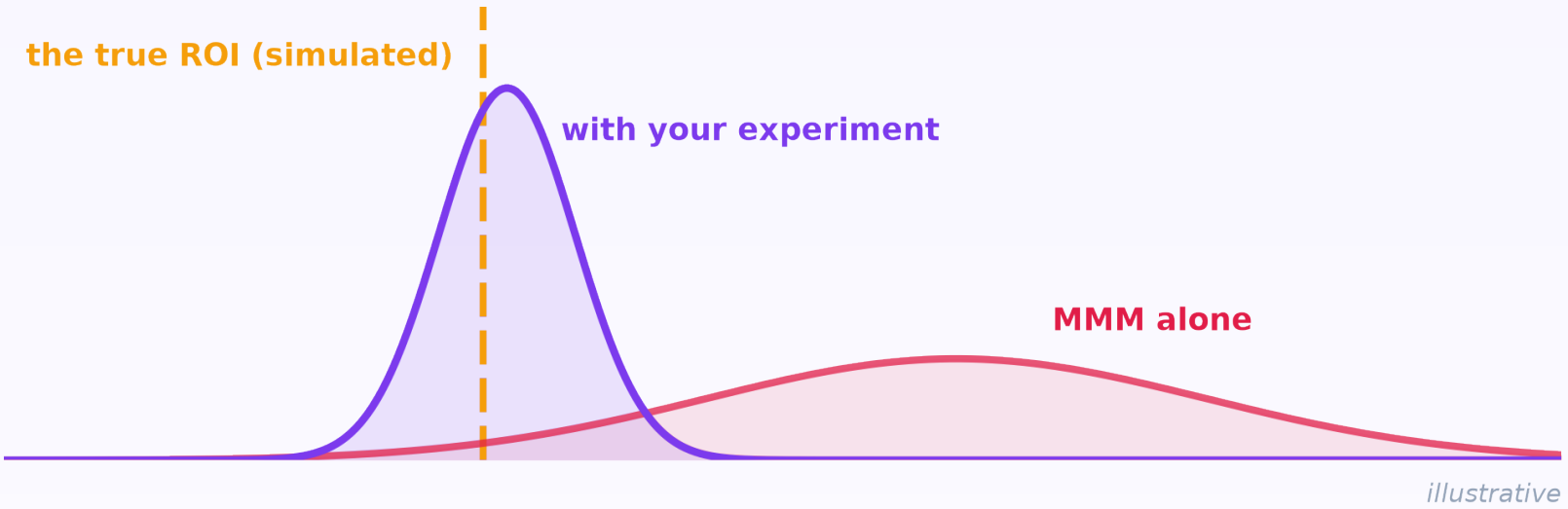
diff - diff

Your geo test is already

an MMM calibration input.



Turn a DiD experiment into your MMM's prior -

in three lines of code.

Speaks Google Meridian + PyMC-Marketing natively.

THE SETUP

You ran the experiment.

Your MMM is still guessing.

A search channel launched in 8 of 12 markets, in three staggered waves. The 4 holdouts never got it. Your MMM has a calibration hook built for exactly this experiment - Meridian takes ROI priors, PyMC-Marketing takes lift tests. Connecting them by hand means deriving, yourself:

a total

incremental outcome, in outcome units

a lognormal

mu / sigma from the estimate and its SE

a mask window

which weeks the prior may speak for

Get any one wrong and the model is

silently mis-calibrated.

Here's the whole bridge.

THE HANDSHAKE

Three lines.

```
res = CallawaySantAnna().fit(panel, outcome='sales', unit='geo',
                             time='week', first_treat='first_treat')

total = res.aggregate('total')      # incremental sales: 184,281

prior = to_meridian_roi_prior(aggregation_result=total,
                              spend=total_spend)   # 74,100


# experiment ROI: 2.487 +/- 0.048
# -> LogNormal(mu=0.9109, sigma=0.0195), ready for Meridian
```

Staggered waves? Handled by the estimator.

Outcome units? The total already speaks them.

Lognormal math? Done, with the SE propagated.

No hand-derived scaling anywhere.

PASTE AND RUN

It writes the

Meridian code for you.

`prior.to_code(channel="search", media_channels=["search", "tv"], roi_calibration_period=mask)`

generated output (abridged - the full snippet builds the mask too):

```
mu = [0.9108575376065811, 0.2]
sigma = [0.019461139350044197, 0.9]
roi_prior = tfp.distributions.LogNormal(mu, sigma, name="roi_m")
prior = prior_distribution.PriorDistribution(roi_m=roi_prior)
model_spec = spec.ModelSpec(
    prior=prior,
    media_prior_type="roi",
    roi_calibration_period=roi_calibration_period,
)
```

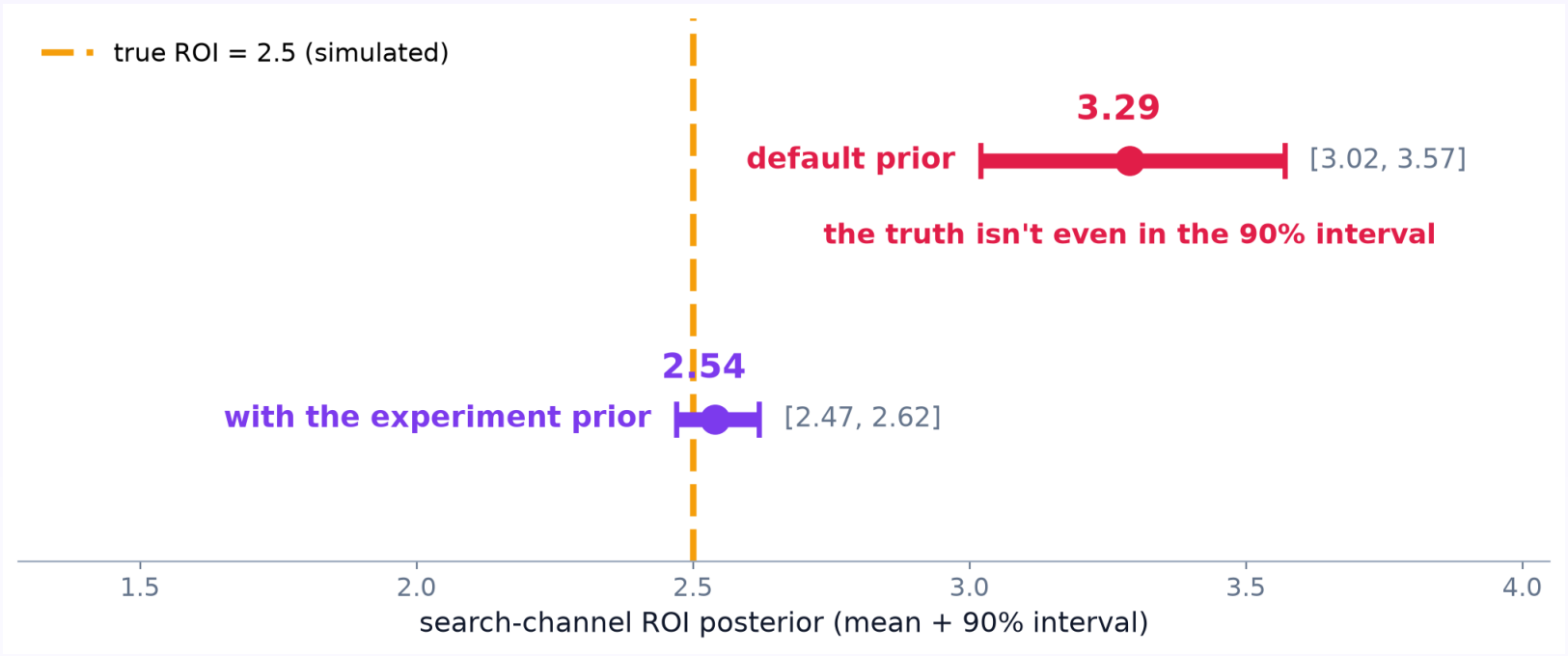
Your experiment's prior on 'search' - the other channels keep Meridian's own defaults.

For 'search', the calibration mask covers the 74-week experiment window and nothing else.

Paste it into your Meridian pipeline - it runs exactly as generated.

THE PAYOFF

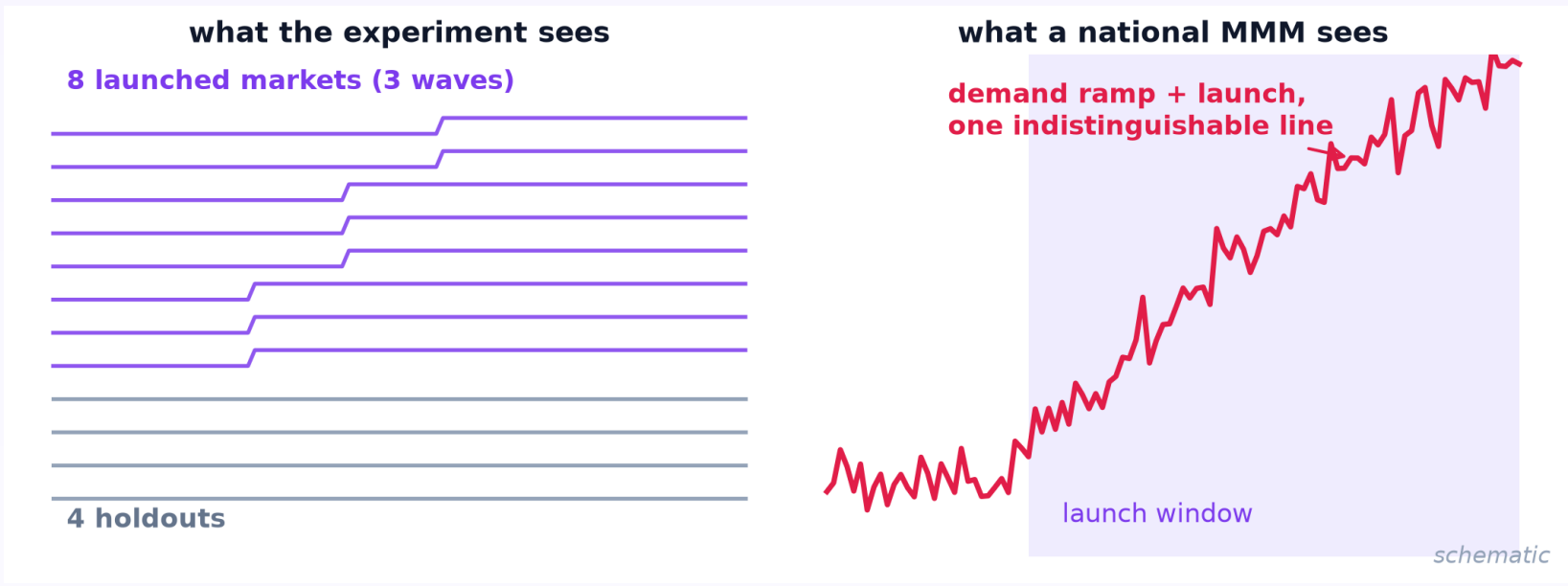
One prior. The posterior snaps to the truth.



With Meridian's default prior, the model settles on 3.29 - confidently, and wrong. Feed it the experiment and the posterior lands on 2.54 against a true ROI of 2.5. Posterior error 0.79 -> 0.04; interval width 0.55 -> 0.15. The DiD measured 2.49 +/- 0.05.

WHY IT WAS WRONG

The national model never saw the contrast.



Aggregation destroys the holdout comparison, and a demand ramp lands right on the launch window - so the model credits the launch. The experiment is the one thing in the data that moved spend independent of demand. That is exactly why both MMMs ship calibration hooks.

RIGOR INCLUDED

The estimand matches,

so you don't have to.

Staggered launches

Callaway-Sant'Anna cohorts - not a naive pre/post average across waves

Launch-from-zero = roi_m

the experiment total IS Meridian's full-spend-vs-none ROI estimand

It checks your setup

malformed windows, wrong signs, double-scaled totals - the easy mistakes fail loudly

Cross-checked estimate

swap in ImputationDiD and the total agrees - a one-line robustness check

You think about the experiment. It thinks about the export.

ALSO SPEAKS PYMC

Spend boost instead?

That's a lift test.

The other classic geo experiment - a budget boost in 9 of 15 markets, with a spend history that chased demand - exports as a PyMC-Marketing lift-test row:

```
df = to_pymc_marketing_lift_test(channel='search', x=1500,
        delta_x=450, aggregation_result=simple, scale=9)
mmm.add_lift_test_measurements(df)   # scale = 9 boosted geos
```

3.52

without the lift test

90% interval [1.78, 5.36]

wide AND centered high

2.21

with it

90% interval [1.91, 2.50]

truth: 2.0

Posterior error 1.52 -> 0.21; the 90% interval is 6x narrower (3.59 -> 0.59). One exported DataFrame did that.

WHAT YOU GET

Built for how you measure.

One-line totals

aggregate('total') on Callaway-Sant'Anna, EfficientDiD, ImputationDiD, TwoStageDiD

Bring your own estimator

synthetic-control geo tests, on/off pulses, dose designs - the explicit route takes your effect + SE

Both MMM dialects

Meridian ROI priors + calibration masks; PyMC-Marketing lift tests, geo dims included

Multiple experiments

spend-weighted pooling into one prior, with optional SE widening

Loud guardrails

wrong-sign and double-scaling mistakes raise - you own the design, it owns the math

sklearn-style API

fit() / summary() / to_dataframe(), like every diff-diff estimator

Zero new dependencies

The exporters emit plain DataFrames, prior parameters, and generated code - your DiD analysis never has to share an environment with Meridian or PyMC. Built against pymc-marketing 1.0 + google-meridian 1.8.

diff - diff

Stop choosing between

the experiment and the model.

```
pip install diff-diff
```

A simulated market, so the truth is known - tutorials 29 and 30 reproduce every number here end to end:

diff-diff.readthedocs.io/en/latest/tutorials/30_mmm_calibration_meridian.html

diff-diff.readthedocs.io/en/latest/tutorials/29_mmm_calibration_pymc.html

github.com/igerber/diff-diff

DiD experiments -> MMM calibration. Now in diff-diff.